
Spring AI and Spring Boot Overview

Introduction

Spring Boot is a popular framework built on top of the Spring ecosystem that simplifies the development of Java applications. It reduces boilerplate configuration and provides production-ready features out of the box. Developers can quickly create standalone applications using embedded servers such as Tomcat, Jetty, or Undertow.

Spring AI is an extension of the Spring ecosystem designed to simplify the integration of Artificial Intelligence models into enterprise applications. It provides abstractions for working with Large Language Models (LLMs), embeddings, vector databases, prompt templates, and Retrieval-Augmented Generation (RAG) architectures.

Modern businesses generate large volumes of documents in formats such as PDF, Word, and HTML. AI applications often need to process these documents and make their contents searchable. Spring AI provides document readers that can ingest content from these sources and convert them into structured documents suitable for AI workflows.

Key Features of Spring Boot

Spring Boot offers several features that improve developer productivity:

1. Auto Configuration
2. Embedded Servers
3. Dependency Management
4. Actuator Monitoring
5. Externalized Configuration
6. Microservice Support

Auto Configuration automatically configures Spring components based on dependencies available in the project. This minimizes manual configuration and allows developers to focus on business logic.

Embedded servers eliminate the need to deploy applications to external application servers. A Spring Boot application can be packaged as a JAR file and executed directly.

Actuator endpoints provide operational insights such as health status, metrics, environment information, and application performance monitoring.

Introduction to Artificial Intelligence

Artificial Intelligence refers to the ability of computer systems to perform tasks that typically require human intelligence. These tasks include natural language understanding, reasoning, image recognition, and decision making.

Large Language Models such as GPT-based systems are trained on massive datasets and can generate human-like responses to user queries. These models are commonly used for chatbots, document analysis, summarization, and content generation.

Organizations are increasingly adopting AI technologies to improve customer service, automate workflows, and gain insights from data. Frameworks such as Spring AI simplify the integration of these capabilities into existing enterprise applications.

Spring AI Document Processing

PDF Document Loading

One of the most common use cases in AI-powered applications is processing PDF documents. Organizations store policies, manuals, reports, invoices, and research papers in PDF format.

Spring AI provides document readers that can extract textual content from PDF files. Once the content is extracted, it can be transformed into document objects and processed further.

The document processing workflow typically consists of the following steps:

1. Read the PDF file.
2. Extract text content.
3. Convert text into document objects.
4. Split large documents into chunks.
5. Generate embeddings.
6. Store embeddings in a vector database.
7. Perform semantic search and retrieval.

This workflow forms the foundation of Retrieval-Augmented Generation systems.

Vector Databases

A vector database stores numerical representations of text called embeddings. Embeddings capture semantic meaning and enable similarity searches.

Popular vector databases include:

- Chroma

- Pinecone
- Weaviate
- Milvus
- PostgreSQL with pgvector

When a user submits a query, the system converts the query into an embedding and searches for similar embeddings in the vector database. The most relevant document chunks are retrieved and supplied to the language model.

This approach significantly improves response accuracy because the model receives context-specific information.

Benefits of Retrieval-Augmented Generation

Retrieval-Augmented Generation offers several advantages:

- Reduced hallucinations
- Improved factual accuracy
- Access to private company documents
- Real-time knowledge updates
- Better transparency

Instead of relying solely on training data, the model retrieves relevant information from external sources before generating a response.

This technique is widely used in enterprise knowledge management systems, customer support applications, and internal search platforms.

Enterprise Use Cases

Many industries are adopting AI-powered document processing solutions.

Healthcare organizations use AI to analyze medical records and research papers.

Financial institutions process reports, compliance documents, and customer statements.

Educational institutions create intelligent search systems for academic content.

Legal firms use AI to analyze contracts, agreements, and regulatory documents.

Government agencies leverage AI to improve document retrieval and citizen services.

As document volumes continue to increase, intelligent retrieval systems become essential for efficient information access.

Future of AI Integration

The future of software development increasingly involves AI-assisted capabilities. Developers are integrating language models into business applications to automate repetitive tasks and enhance user experiences.

Spring AI provides a consistent programming model for interacting with AI services while maintaining the familiar Spring development experience.

As AI technology evolves, frameworks such as Spring AI will continue to simplify adoption and enable organizations to build intelligent applications at scale.

The combination of Spring Boot, Spring AI, vector databases, and Retrieval-Augmented Generation creates a powerful platform for developing next-generation enterprise solutions.

Example JUnit Assertions for Testing

After loading the PDF:

```
List<Document> docs = pdfService.loadDataFromPdf("sample.pdf");


assertNotNull(docs);

assertFalse(docs.isEmpty());


String content = docs.stream()

    .map(Document::getText)

    .collect(Collectors.joining(" "));


assertTrue(content.contains("Spring Boot"));

assertTrue(content.contains("Spring AI"));

assertTrue(content.contains("Vector Databases"));

assertTrue(content.contains("Retrieval-Augmented Generation"));
```

This sample text is roughly **900–1200 words**, which usually becomes **2–3 PDF pages** depending on font size and spacing, making it suitable for testing loadDataFromPdf() and document chunking in Spring AI.